

OSWG

Oddly Specific Wordlist Generator

Version 0.7.0

User Guide

Repository: <https://github.com/dmarakom6/oswg>

Version: 0.7.0

License: MIT

Homebrew tap: [dmarakom6/homebrew-oswg](#)

Docker image: [ghcr.io/dmarakom6/oswg](#)

Generated: September 24, 2026

Official documentation - generated from the repository.

1 Introduction

OSWG (Oddly Specific Wordlist Generator) builds **targeted wordlists** from a website's own content. Instead of shipping a generic dictionary, it crawls the site you give it, extracts the words that actually matter - **keywords**, **usernames**, and **email addresses** - and applies a mutation engine that expands them into thousands of realistic guesses: leetspeak variants, case changes, number suffixes, special characters, and random combinations.

OSWG is a Python CLI plus a SvelteKit **web dashboard** in a single package. The dashboard mirrors the CLI (`oswg ui`) and adds real-time progress, visualizations, and saved templates. A test command drives external crackers (hashcat, john, hydra, medusa, ncrack, gobuster, aircrack-ng) against your generated wordlist, so you can measure how well it performs before you deploy it.

What OSWG does

- Crawl - a polite crawler follows links (or sitemap.xml) and collects page text, titles, and metadata.
- Extract - keywords, email addresses, and usernames are pulled from the scraped content.
- Mutate - the mutation engine applies leet-speak, case, number-suffix, and special-character transforms.
- Generate - mutations are deduplicated, filtered, and trimmed to a target wordlist size.
- Dashboard - the web UI runs the same engine with live progress and visualizations.
- Test - external crackers are run against the finished wordlist and hits are summarized.

2 Installation

OSWG can be installed with pip, as a prebuilt binary, via Homebrew, from a .deb package, or run as a Docker image. All install methods produce the same `oswg` command.

2.1 pip

```
pip install oswg
```

Optional extras are installed with extras syntax: `oswg[auth]` adds NTLM HTTP authentication

```
pip install 'oswg[auth]'  
pip install 'oswg[js]'
```

After installation, `oswg setup` reports which optional components are missing and offers to install

```
oswg setup          # show missing extras, install interactively  
oswg setup --check  # status only, install nothing
```

2.2 Prebuilt binaries (GitHub Releases)

Each GitHub Release ships single-file, dependency-free binaries for three platforms. Download the right one from <https://github.com/dmarakom6/oswg/releases>, make it executable, and run it

Platform	Binary
Linux x86_64	oswg-linux-x86_64
macOS x86_64	oswg-macos-x86_64
macOS Apple Silicon (arm64)	oswg-macos-arm64

```
chmod +x oswg-linux-x86_64  
./oswg-linux-x86_64 ui
```

2.3 Debian / Ubuntu (.deb)

Download `oswg_<version>_amd64.deb` from the latest release and install it with `apt`. The package

```
sudo apt install ./oswg_0.7.0_amd64.deb
```

```
sudo apt remove oswg
```

2.4 Homebrew (macOS)

```
brew tap dmarakom6/oswg  
brew install oswg
```

2.5 Docker (self-hosted UI)

For a self-hosted dashboard, run the published image. It bundles Chromium, so JavaScript rendering works out of the box - no `oswg` setup needed. Jobs, templates, and generated wordlists persist in the

```
docker run -d --name oswg -p 8000:8000 -v oswg-data:/data ghcr.io/dmarakom6/oswg
```

Cracking tools used by `oswg test` stay **host-side**: the container runs the CLI and dashboard, while hashcat, John, hydra, and friends run on your own machine against targets you point it at.

3 CLI reference

All functionality is exposed through a single `oswg` binary. Run `oswg --help` for the full command list and `oswg <command> --help` for per-command flags. Every command prints a human-readable summary with rich formatting.

3.1 generate - build a wordlist

```
oswg generate <url>
```

Crawls the target, extracts keywords, and applies the mutation engine to produce a wordlist. Accepts

Flag	Meaning
<code>--preset</code> <code>quick standard aggressive extreme</code>	Apply a built-in preset (wordlist size plus advanced options).
<code>--template <name></code>	Load a saved template as the base configuration.
<code>--sitemap</code>	Discover pages from <code>sitemap.xml</code> .
<code>--js-render</code>	Render pages with a real browser (requires <code>oswg[js]</code>).
<code>--emails</code>	Include email addresses found on the target.
<code>--username</code>	Extract usernames to a sidecar file (never merged in).
<code>--ai-completions</code>	Expand base words with AI - Ollama locally, or OpenAI with <code>OPENAI_API_KEY</code> .
<code>--output, -o <file></code>	Output path (default: <code>wordlist.txt</code>).
<code>--format txt json csv</code>	Output format; inferred from the file extension.
<code>--gzip / --no-gzip</code>	Gzip-compress the output.

Flag	Meaning
--dry-run	Preview the wordlist without writing a file.
--merge <file>	Merge an external wordlist file (repeatable).
--merge-builtin	Merge bundled common passwords.
--merge-rockyou	Merge /usr/share/wordlists/rockyou.txt(.gz) if present.
--leet-level 1 2	Leet-speak intensity (1 = basic, 2 = advanced).
--random-combine	Bind random pairs of base words (e.g. NutellaCream2024).
--combine-count <n>	Number of random combinations (default: 1000).

```
oswg generate https://example.com --preset aggressive --emails -o example.txt
```

3.2 scrape - extract keywords

oswg scrape <url>

Extracts keywords from a target without generating a wordlist - useful for recon before building a

Flag	Meaning
--output, -o <file>	Save keywords to a file (preview on stdout if omitted).
--sitemap	Use sitemap.xml for page discovery.
--js-render	Render pages with a real browser.
--emails	Include email addresses found on the target.
--username	Extract usernames to a sidecar file.
--rate-limit <sec>	Delay between requests in seconds.
--jitter	Randomize the delay by +/- 50% (with --rate-limit).
--respect-robots	Honor robots.txt rules.
--user-agent <ua>	Custom User-Agent header for requests.

```
oswg scrape https://example.com --emails --rate-limit 1 -o keywords.txt
```

3.3 mutate - apply the mutation engine

oswg mutate <file>

Applies the same mutation engine used by generate to an existing word file (one word per line) or to

```
oswg mutate base-words.txt --leet-level 2 --special -o mutations.txt
```

Shared mutation flags include --leet-level, --no-leet, --no-numbers, --special, --append, --prepend, --case-permutations, and --random-combine.

3.4 test - drive an external cracker

```
oswg test <wordlist> --tool <tool>
```

wordlist Generator

Runs an external cracking tool against the wordlist and summarizes any hits. The tool itself must be

Tool	Key flags	Use case
hashcat	--hashes <file>, --mode <num>	Crack a hash file (mode 0 = MD5, 1000 = NTLM).
john	--hashes <file>	Crack a hash file with John the Ripper.
hydra	--url, --users, --passwords	Online login brute force.
medusa	--url, --users, --passwords	Online login brute force.
ncrack	--url, --users, --passwords	Network authentication cracking.
gobuster	--url	Directory and file brute force.
aircrack	--capture <pcap>	Crack a Wi-Fi capture with aircrack-ng.

```
oswg test wl.txt --tool hashcat --hashes hashes.txt --mode 0
oswg test paths.txt --tool gobuster --url http://127.0.0.1:9202/
```

3.5 template - manage saved job templates

```
oswg template list | save NAME --type generate|scrape [--from-job <job_id> |
--config-file <file>] | delete NAME
```

Manage saved job templates so a good configuration can be reused.

- `oswg template list` - list saved templates with their type and creation time.
- `oswg template save NAME --type generate --from-job <job\_id>` - save a completed job's configuration.
- `oswg template save NAME --type scrape --config-file <file>` - save from a JSON config file.
- `oswg template delete NAME` - remove a saved template.

```
oswg template save intranet-audit --type generate --from-job 8f6c2a1e-...
oswg template list
oswg template delete intranet-audit
```

3.6 jobs - list recent jobs

```
oswg jobs
```

Lists recent jobs with their ID, type, status, and target URL - useful when you missed a job ID in the output of generate or scrape.

3.7 login - capture a browser session

```
oswg login <url>
```

Opens an interactive browser so you can log in to a site, then captures the authenticated session (a Playwright storage-state JSON) for reuse with --session-file on generate and scrape.

3.8 ui - launch the web dashboard

```
oswg ui
```

Starts the FastAPI backend and opens the web dashboard in your browser at `http://127.0.0.1:8000` (the port auto-increments when busy).

Flag	Meaning
--host <addr>	Bind address (default: 127.0.0.1).
--port <num>	Base port (default: 8000).
--no-browser	Do not open the browser automatically.

```
oswg ui
oswg ui --host 0.0.0.0 --port 8080 --no-browser
```

3.9 Job IDs

oswg generate and oswg scrape persist every run as a **job** and print its identifier when the run

```
Job ID: 8f6c2a1e-3b4d-4e5f-9a0b-1c2d3e4f5a6b
```

```
oswg template save intranet-audit --type generate --from-job 8f6c2a1e-3b4d-4e5f-9a0b-1c2d3e4f5a6b
```

If you missed the ID, oswg jobs lists the most recent runs.

4 Web dashboard

oswg ui opens the dashboard with four tabs - **Generate**, **Scrape**, **Mutate**, and **Test** - mirroring the CLI commands. Each tab collects a form; submissions stream progress in real time over a WebSocket and appear in the job history.

4.1 Templates and presets

Below the URL field sits a collapsed **Templates & Presets** section:

- Presets (quick, standard, aggressive, extreme) set both the target wordlist size and the advanced options - leet level, special characters, random combinations, merges, and minimum/maximum word length.
- Applying a preset or a saved template disables the other dropdown, so the two cannot conflict.
- Saved templates are also created from the ****Save**** button next to ****Visualize**** on any completed job.
- ****Clear all**** deletes every saved template in one click.

4.2 Visualizations

Completed jobs can be explored with four visualizations:

- Coverage map - a sunburst chart of the crawled site structure.
- Keyword cloud - the extracted keywords sized by frequency.
- Frequency chart - the word-frequency distribution of the generated list.
- Mutation tree - how each base word expanded into its mutations.

4.3 Notifications and keyboard shortcuts

- Browser notifications - enable the bell toggle to be notified when a job completes.
- Keys 1-4 switch tabs, and `f` focuses the URL field.
- `?` opens the shortcuts help overlay.
- mod+Enter (Ctrl/Cmd+Enter) runs the current form.

4.4 Data location

OSWG follows the XDG Base Directory Specification. Jobs, templates, and generated wordlists live in ~/.local/share/oswg. Override with the XDG_DATA_HOME or OSWG_DATA_DIR environment variables.

5 Worked examples

5.1 Gobuster directory brute force

A local site is served at `http://127.0.0.1:9202/` and exposes pages like `/index.html` and `/p2.html`, both answering HTTP 200. Generate a small path-focused wordlist from the site, then run

```
oswg generate http://127.0.0.1:9202/ --preset quick -o paths.txt
oswg test paths.txt --tool gobuster --url http://127.0.0.1:9202/
```

Path	Status
/index.html	200
/p2.html	200

5.2 Hashcat MD5 crack

A wordlist `wl.txt` contains the word `password`, and a hashes file `hashes.txt` contains the MD5 digest `5f4dcc3b5aa765d61d8327deb882cf99` (the MD5 of `password`). Crack it with hashcat in

```
oswg test wl.txt --tool hashcat --hashes hashes.txt --mode 0
```

Hash	Password
5f4dcc3b5aa765d61d8327deb882cf99	password

These examples are for authorized testing only. Run OSWG and the external crackers only against systems you own or have explicit permission to test.